

Libraries

2026 Semester 2 COMPSCI 340: Operating Systems
Talía Xu

Lecture 3
2.1.0

Based on original slides by Professor Jon Eyolfson.

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

Libraries Connect Applications to Operating-System Services

Recall: system calls are the controlled interface into the kernel.

Most application code calls **libraries**, which may compute entirely in user space or eventually make system calls.

Today we will connect four ideas:

1. the layers that make up an operating system,
2. compilation, linking, and loading,
3. static and dynamic libraries, and
4. API and ABI compatibility.

An Operating System Is More Than Its Kernel

Kernel

- Protects hardware
- Manages shared resources
- Provides system calls

System libraries

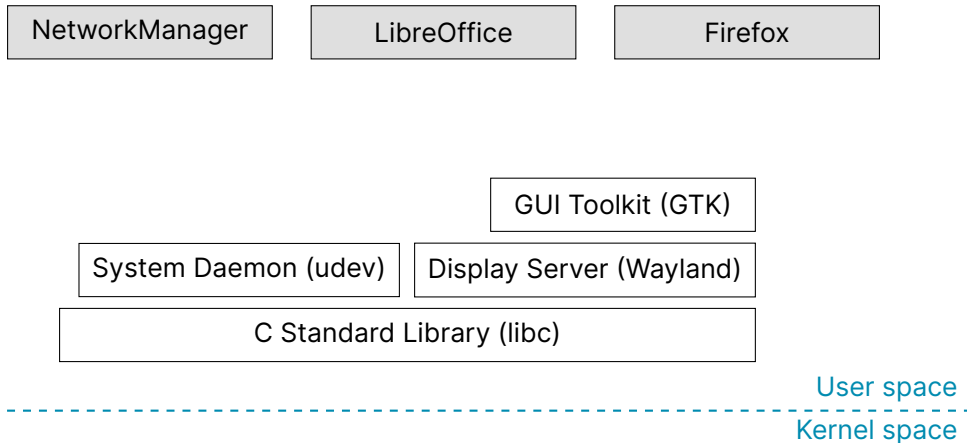
- Provide reusable functions
- Hide low-level details
- Define common interfaces

System services

- Manage devices and networks
- Provide display and login services
- Support applications

For an application, the useful “operating system” is the complete set of interfaces it relies on.

An Application May Pass Through Several User-Space Layers



A library call may finish in user space or eventually request a kernel service.

The Application Determines Which OS Layers Matter

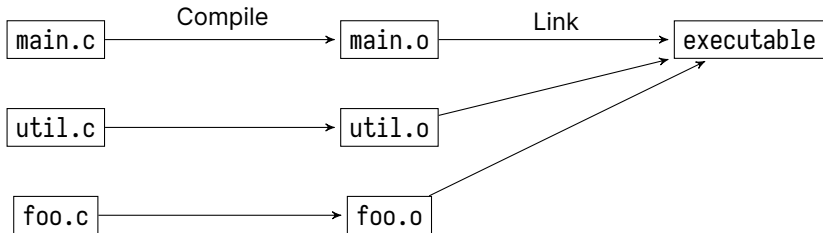
Android and Debian both use the Linux kernel, but provide different libraries, services, and application environments.

A terminal program may run on both when the interfaces it needs are available.

A graphical Android application expects Android-specific frameworks and will not run on Debian merely because the kernel is Linux.

Compatibility depends on the whole interface stack, not only the kernel.

Compilation and Linking Solve Different Problems



The compiler translates one source file. The linker combines object files and resolves references between them.

Symbols Let Separately Compiled Files Meet

main.c uses a symbol

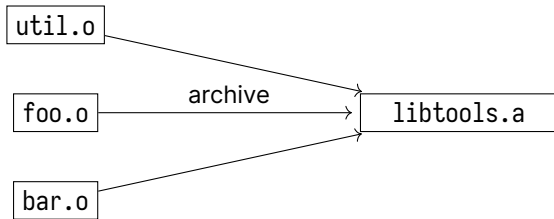
```
int add(int, int);  
  
int main(void) {  
    return add(2, 3);  
}
```

util.c defines it

```
int add(int x, int y) {  
    return x + y;  
}
```

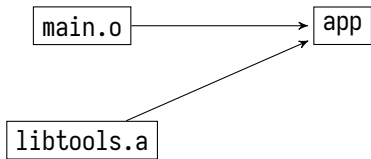
Object files contain machine code plus symbol and relocation information. The linker matches the unresolved add reference to its definition.

A Static Library Is an Archive of Object Files



The archive packages reusable object files. At link time, the linker extracts the members needed to resolve the program's symbols.

Static Linking Places Library Code in the Executable



Benefits

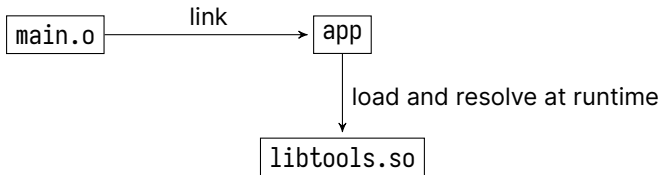
- Fewer runtime dependencies
- Predictable deployed version

Costs

- Larger executables and duplication
- Re-link to receive library fixes

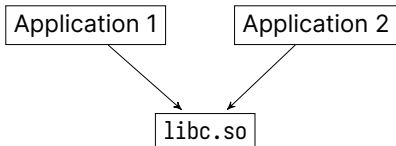
The executable carries the selected library code with it.

Dynamic Linking Keeps a Runtime Dependency



The executable records which shared libraries and symbols it needs. The dynamic loader finds the libraries, maps them, and resolves those symbols.

Shared Libraries Reduce Duplicate Code in Memory



- Each process has a virtual-memory mapping for the library
- Read-only code pages can share physical memory
- Writable state remains private to each process

Sharing is a memory benefit; compatibility is the price of replacing the library independently.

ldd Reveals an Executable's Shared-Library Dependencies

On Linux:

```
$ ldd ./app
libtools.so => /usr/lib/libtools.so (...)
libc.so.6 => /usr/lib/libc.so.6 (...)
ld-linux-x86-64.so.2 => /usr/lib/ld-linux-x86-64.so.2 (...)
```

This output shows which library file satisfies each recorded dependency.

If a required library cannot be found, the program fails before main begins.

Static and Dynamic Linking Make Different Trade-offs

	Static library	Dynamic library
Executable	Includes selected code	Needs library at runtime
Sharing	Duplicates code	Code pages may be shared
Deployment	Mostly self-contained	Library must be installed
Updates	Re-link executable	Replace library
Main risk	Embedded code becomes stale	Missing or incompatible library

Neither is always better: the choice depends on deployment, update, memory, and compatibility requirements.

API and ABI Are Different Contracts

API: source-level contract

- Function names and types
- Constants and data types
- Documented behaviour
- Used when compiling source

ABI: binary-level contract

- Symbol names
- Calling convention
- Type size and layout
- Used by linker and loader

Source code can still compile against a new API while an already compiled executable is incompatible with the new ABI.

A Public struct Layout Becomes Part of the ABI

Library v1

```
struct point {  
    int y;  
    int x;  
};
```

x is at offset 4.

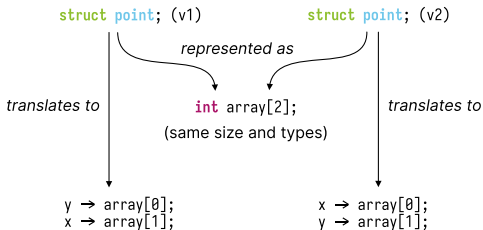
Library v2

```
struct point {  
    int x;  
    int y;  
};
```

x is at offset 0.

The field names are unchanged, but compiled field accesses use different byte offsets.

The Compiler Turns Field Access into Fixed Offsets



The executable remembers the layout from the header used at compile time; it does not rediscover field locations at runtime.

An API Can Still Expose ABI-Sensitive Representation

libpoint.so

```
struct point *point_create(int x, int y);  
int point_get_x(struct point *p);  
int point_get_y(struct point *p);  
void point_destroy(struct point *p);
```

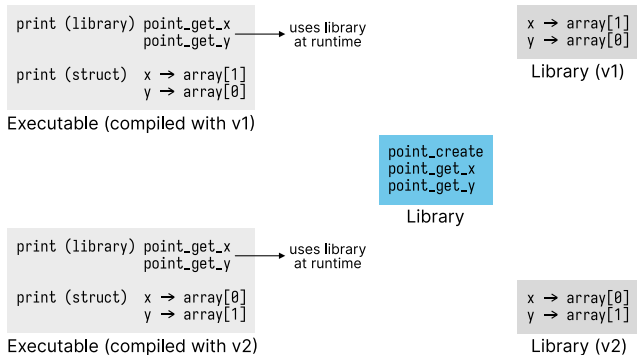
Functions such as `point_get_x` hide the field access inside the library. Exposing `struct point` also lets client code compile direct accesses such as `p->x`.

One Program Can Access the Same Object in Two Ways

```
int main(void) {  
    struct point *p = point_create(1, 2);  
  
    printf("point (x, y) = %d, %d (using library)\n",  
          point_get_x(p), point_get_y(p));  
  
    printf("point (x, y) = %d, %d (using struct)\n", p->x, p->y);  
  
    point_destroy(p);  
    return 0;  
}
```

The first printf asks library functions for the fields. The second uses offsets compiled directly into the executable.

Mismatched Versions Disagree on Where x and y Live



With a v1 executable and v2 library, library functions use the v2 layout while direct field access still uses the v1 offsets.

Opaque Types Keep Private Layout Out of the ABI

Public header

```
struct point;  
  
struct point *point_create(int x,  
                           int y);  
int point_get_x(struct point *p);  
int point_get_y(struct point *p);  
void point_destroy(struct point *p);
```

Library implementation

```
struct point {  
    int x;  
    int y;  
};
```

Client code cannot compile `p->x`; it must use the stable functions.

Hiding representation lets the library change its internals without changing the client-visible binary layout.

Check the Contract Before Predicting Compatibility

An executable was compiled with the v1 header, then loads the v2 library. The function signatures are unchanged, but struct point was reordered.

Ask in order:

1. Did the source-level API change?
2. Did any binary-level layout or calling rule change?
3. Does the executable access the representation directly?

Here the API looks stable, but the ABI is not. Calls that cross the version boundary may disagree about the object's layout.

Summary: Libraries Add Reuse and a Compatibility Contract

- Applications rely on a stack of libraries and services above the kernel
- Compilation produces object files; linking resolves symbols into a program
- Static linking embeds selected code; dynamic linking resolves a shared library at runtime
- APIs describe source-level use, while ABIs describe binary-level agreement
- Public data layouts can break old executables; opaque types reduce that risk

Next: the operating system turns an executable program into a running process.